Parallel Image Stitching

Davide Lombardi
E-mail address
davide.lombardi2@edu.unifi.it

Abstract

Image stitching is a computer vision task that combines multiple images with overlapping fields of view to produce a high-resolution panorama. However, the standard sequential pipeline (that comprises feature extraction, matching, homography estimation, warping, and re-extraction) is computationally expensive and scales poorly with large image sets. This project explores the parallelization of the image stitching pipeline in Python, evaluating multiple concurrency paradigms to overcome language-specific bottlenecks such as the Global Interpreter Lock (GIL) and Inter-Process Communication (IPC) serialization overhead. Several parallel architectures, including Task Parallelism and Data Parallelism, were implemented and benchmarked against a sequential baseline. Experimental results indicate that while isolated phases like SIFT extraction scale efficiently (achieving up to a 2.8x speedup using process pools), the overall aggregate performance is heavily constrained by strict sequential dependencies, primarily the feature re-extraction phase on the continuously updating panorama.

1. Introduction

Image stitching is an important process in computer vision, widely utilized to construct high-resolution panoramic images from a sequence of overlapping photographs. This technique finds extensive application in fields such as geospatial mapping, autonomous driving, virtual reality, and consumer photography. Despite its widespread adoption, the creation of a seamless panorama from multiple independent images demands substantial computational resources. Each stage of the pipeline introduces unique mathematical and algorithmic challenges that scale poorly when processed sequentially, particularly as image resolutions and dataset sizes increase.

The main objective of this work is to analyze the performance bottlenecks inherent to the standard sequential image stitching pipeline and evaluate the efficacy of various parallelization strategies in Python. This section delineates the theoretical foundations and functional role of each phase within the stitching framework.

1.1. Feature Extraction

The initial phase of the pipeline involves identifying distinctive points within each input image that remain invariant to scale, rotation, and illumination changes. In this framework, the Scale-Invariant Feature Transform (SIFT) algorithm is employed. SIFT detects local extrema in scale-space by computing the Difference of Gaussians (DoG), defined mathematically as $D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$, where L is the image convolved with a Gaussian blur. For each keypoint, SIFT then computes a 128-dimensional descriptor. Because feature extraction operates independently on each discrete image file, this phase represents an ideal candidate for data-level parallelization across multiple worker processes.

1.2. Feature Matching

Once keypoints and descriptors are computed for the input images, correspondence must be established between adjacent frames. Feature matching computes the distance (typically us-

ing the L2 norm for SIFT) between descriptors in different images to identify overlapping regions. This implementation utilizes a Fast Library for Approximate Nearest Neighbors (FLANN) matcher combined with Lowe's ratio test to filter out ambiguous and spurious matches. Formally, a descriptor d_A is matched to its nearest neighbor d_{B1} only if the ratio of distances to the first and second nearest neighbors (d_{B2}) satisfies $\frac{||d_A - d_{B1}||_2}{||d_A - d_{B2}||_2} < \tau$, with τ typically set around 0.7.

1.3. Homography Estimation

The next step involves aligning the matched keypoints from the source image to the target image. This is achieved by estimating a 3×3 homography matrix (H) using the Random Sample Consensus (RANSAC) algorithm. Mathematically, H projects a keypoint $p = [x, y, 1]^T$ from the source image to the target coordinate space $p' = [x', y', 1]^T$ via the linear equation $p' \sim Hp$, where the equality is up to a non-zero scale factor. RANSAC iteratively selects a minimal subset of matched points to estimate the transformation model, separating valid inliers from outlier matches caused by false correspondences. This phase is characterized by an iterative, non-deterministic execution time that depends heavily on the quality of the initial feature matches.

1.4. Warping and Blending

After determining the homography matrices, the target images are transformed (warped) into the spatial domain of the reference frame or the evolving canvas. Warping requires computing backward coordinate mapping and pixel interpolation. To prevent visible seams, exposure mismatches, and ghosting artifacts in the overlapping regions, a blending mask or multi-band blending algorithm is applied. In a standard linear alpha blending, the final pixel intensity is computed as $I_{blend}(x, y) = \alpha(x, y) I_{ref}(x, y) + (1 - \alpha(x, y)) I_{warped}(x, y)$, where $\alpha \in [0, 1]$ dictates the pixel contribution. This phase involves heavy matrix operations and pixel-level manipulation, making memory bandwidth and array serialization significant factors during parallel execution.

1.5. Feature Re-extraction (Re-ext)

In a linear or progressive stitching pipeline, a severe structural bottleneck arises when appending subsequent frames. Once an image is warped and blended onto the main canvas, the existing keypoint coordinate system is altered, and new overlapping regions are created. Then, features must be re-extracted from the updated panorama to facilitate matching with the next incoming frame. The computational cost of this phase escalates dramatically, because the dimensions of the canvas grow progressively with each integrated image. If the initial image area is A_0, the panorama area after n iterations expands to $A_n \approx A_0 + \sum_{i=1}^{n} \Delta A_i$, causing the time complexity of re-extraction to monotonically increase over time. Moreover, strict data dependencies require the completion of the current blending step before the next re-extraction can begin, imposing a rigid sequential constraint on the architecture.

1.6. Pipeline Execution Overview

To illustrate the complete workflow, the framework processes a sequential dataset of multiple overlapping images and aligns them into a single coherent panorama. The structural transformation from the independent inputs to the final synthesized output is shown in Figure **??**.

2. Implementation Details

This section details the experimental environment, the preprocessing strategy applied to the drone imagery dataset, and the specific concurrent architectures designed to accelerate the image stitching pipeline in Python.

2.1. Dataset and Preprocessing Setup

For the experimental validation of the framework, the OpenDroneMap dataset [1] is used, which contains high-resolution aerial imagery (4896 × 3672 pixels) captured via UAVs. All images are downsampled to 50% of their original resolution: this choice is strictly dictated by the

necessity to compress the memory footprint and bound the initial SIFT computational complexity.

Progressive image stitching faces a major issue when processing frames in strict chronological order. If two consecutive images fail to align (usually due to low-texture areas or sharp changes in perspective), the main canvas stops updating. As a result, the system prematurely drops all remaining images, even if they share valid overlaps. To resolve this weakness without changing the core stitching algorithm, a separate diagnostic and optimization framework using `window_diagnostics.py` and `reorder_windows.py` is employed.

Operating within a localized sliding execution window, this preprocessing routine executes a brute-force pairwise analysis. Features are extracted once per image to compute the RANSAC inlier count for all possible $C(n, 2)$ combinations, mapping the visual connectivity matrix of the local subset. Through this heuristic analysis, the algorithm identifies the "hub" image, defined as the frame exhibiting the highest degree of valid correspondences and total inlier density. Consequently, files are physically reordered and renamed on disk. This layout ensures that each sliding window initiates its progressive blending sequence from its optimal structural anchor, preventing recoverable topological data from being lost due to localized tracking loss.

2.2. Parallelization Scope and System Constraints

The design of an effective concurrent architecture in Python is strictly governed by loop-carried dependencies and the rigid execution bottlenecks of the CPython interpreter. Driven by the fundamentally distinct computational demands of each processing stage, the achievable parallelization scope shifts considerably throughout the pipeline:

- **Parallelizable phases:** loading images is an I/O-bound task, making it a perfect fit for multithreading. Since Python releases the GIL during file operations, multiple threads can read from disk at the same time without blocking each other. By contrast, SIFT feature extraction is a heavy CPU-bound task: since features are computed independently for each image, this step is highly parallel. A pool of worker *processes* allows the system to completely bypass the GIL instead of just dealing with it. Finally, the warping and blending stage is parallelized differently: rather than splitting the workload per image, the output canvas is divided into horizontal tiles. These tiles are blended concurrently, exploiting data parallelism while merging two images.
- **Non-parallelizable phases (sequential bottleneck):** feature matching, homography estimation (RANSAC), and canvas blending form a strictly sequential chain. Each stitch operates on the panorama produced by the previous one, so stitch i cannot begin before stitch $i - 1$ has committed its result to the canvas. Feature re-extraction inherits this constraint, since it must run on the newly merged canvas rather than on the original input images. Given that the canvas grows monotonically with each merge , this re-extraction step becomes an increasingly expensive, strictly serial bottleneck that cannot be parallelized within a simple left-to-right (linear-fold) alignment strategy, motivating the alternative merge strategies discussed later (e.g. the tree-based MapReduce fold).

To systematically evaluate the trade-offs between concurrency paradigms, language-level overheads, and synchronization costs, six distinct architectural configurations were implemented and benchmarked against each other.

2.3. Sequential Baseline (`sequential.py`)

This configuration serves as the experimental control: the entire stitching pipeline (image loading, SIFT extraction, matching, homography estimation, warping, blending, and re-extraction) runs on a single thread within a single process, with no concurrency primitives of any kind. Its purpose is not performance but *measurement*:

by isolating the raw, unparallelized cost of each phase, it establishes the reference timings against which every other architecture's speedup is computed.

Concretely, this script processes the workload in two distinct stages: it first extracts SIFT features for all input images upfront, and then initiates the iterative left-to-right blending loop. This implementation serves as a direct practical manifestation of the progressive re-extraction bottleneck described previously.

Listing 1: Core sequential fold in `sequential.py`: each iteration strictly depends on the panorama state produced by the previous one.

```
base_image = images[0]
base_kp, base_des = kp_list[0], des_list[0]

for i in range(1, len(images)):
    matches = match_features(base_des,
        des_list[i])
    if len(matches) < 4:
        continue

    H = estimate_homography(base_kp,
        kp_list[i], matches)
    if H is None:
        continue

    base_image =
        warp_and_blend(base_image,
        images[i], H)

    gray = cv2.cvtColor(base_image,
        cv2.COLOR_BGR2GRAY)
    base_kp, base_des =
        sift.detectAndCompute(gray, None)
```

Every other pipeline evaluated in this study preserves this same match $\rightarrow$ homography $\rightarrow$ warp $\rightarrow$ re-extract dependency chain unchanged; what differs between architectures is exclusively *how* the independent, non-sequential work around this chain is scheduled onto threads or processes.

2.4. Parallel Pipeline (`parallel.py`)

This configuration uses a targeted approach to concurrency. Instead of applying a single parallel method to the entire pipeline, it addresses three specific bottlenecks independently. The choice between using threads or processes is decided case by case, depending on whether the task is I/O-bound or CPU-bound, and whether the GIL restricts its performance.

Threaded I/O loading. Image loading is handled by a `ThreadPoolExecutor`. Since `cv2.imread` mostly waits for the disk, Python releases the GIL during each read operation. This allows multiple threads to load files at the same time without blocking each other. Multiple processes are not needed here because reading files is an I/O-bound task, not a heavy CPU computation.

Process-pool SIFT extraction. SIFT feature extraction is a heavy CPU-bound task. Using threads here would not help because Python's GIL prevents true parallel execution across multiple images, then a `ProcessPoolExecutor` is used. This gives each worker its own Python interpreter, bypassing the GIL completely. However, this creates a data transfer issue: `multiprocessing` uses "pickling" to serialize and move data between processes, but OpenCV `cv2.KeyPoint` objects cannot be pickled natively. To fix this, each worker breaks down the keypoints into simple Python tuples before returning them, and the main process rebuilds them into `cv2.KeyPoint` objects later. Finally, since spawning new OS processes is slow, the process pool is created only once at the beginning and reused for all windows.

Tiled blending. The final stage uses a third type of parallelism. Instead of parallelizing across images or I/O requests, the blending step parallelizes within a single merge operation. Because alpha-blending relies on NumPy array arithmetic, Python releases the GIL during these vectorized operations. Therefore, a `ThreadPoolExecutor` is highly efficient and avoids the high overhead of processes, even though the computation is numerically heavy. The canvas is divided into horizontal tiles based

on the number of available CPU cores. Each tile is blended independently by a thread, and the final results are reassembled vertically using `np.vstack`. Additionally, this stage protects against bad homography matrices. Before allocating memory, the expected canvas size is checked against a set limit of the combined input area. If the size is too large, an error is raised early to prevent out-of-memory crashes caused by an unstable RANSAC result.

Crucially, these optimizations do not change the core sequential dependency chain (matching, homography, warping, and re-extraction). This chain remains strictly serial because tiling only parallelizes a single merge operation internally; it does not allow multiple separate merges to run at the same time.

Listing 2: Process-boundary serialization workaround for `cv2.KeyPoint` in `parallel.py`: each worker returns primitive tuples, which the parent process reconstructs into native OpenCV objects.

```
def extract_single_image_features(img):
    cv2.setNumThreads(1)  # avoid
        oversubscription across worker
        processes
    sift = cv2.SIFT_create(nfeatures=8000)
    gray = cv2.cvtColor(img,
        cv2.COLOR_BGR2GRAY)
    kp, des = sift.detectAndCompute(gray,
        None)

    # cv2.KeyPoint is a C++ binding and
        cannot be pickled directly;
    # decompose it into a tuple of
        primitives for cross-process
        transfer.
    kp_serialized = [
        (p.pt, p.size, p.angle,
            p.response, p.octave,
            p.class_id)
        for p in kp
    ]
    return kp_serialized, des

# Back in the parent process
results =
    list(process_executor.map(extract_single_image_features,
    images))
keypoints_list = [
    [cv2.KeyPoint(pt[0], pt[1], size,
        angle, response, octave, class_id)
     for pt, size, angle, response,
         octave, class_id in kp_serialized]
    for kp_serialized, _ in results
]
```

2.5. Producer-Consumer Paradigm

This variant decouples the two structurally different workloads present in the stitching pipeline: SIFT feature extraction, which has no dependency on the state of the growing panorama, and the sequential fold chain (matching $\rightarrow$ homography estimation $\rightarrow$ warping $\rightarrow$ re-extraction), whose steps are strictly ordered because each stitch operates on the panorama produced by the previous one. Rather than executing these two workloads back-to-back, the producer-consumer pattern overlaps them in time, so that feature extraction for image $i + 1$ proceeds concurrently with the fold operations being applied to image i.

Architecture. A single *producer* thread submits every image's feature-extraction job to a warm `ProcessPoolExecutor` at the start of the window, allowing the pool to schedule extraction across all available cores. As each future resolves, the producer deserialises the returned keypoints and pushes the tuple $(\text{index}, \text{image}, \text{keypoints}, \text{descriptors})$ onto a bounded `queue.Queue`, strictly in image order. A *consumer*, running on the main thread, blocks on this queue and, as soon as an item becomes available, drives the sequential fold.

It is worth underlining that producer and consumer live in the *same* operating-system process, communicating through a standard `queue.Queue` rather than a `multiprocessing.Queue`. Because both ends share the same address space, handing an item from producer to consumer only requires passing an object reference, with no serialisation cost. Serialisation (pickling) is incurred exactly once per image, only at the process boundary between the producer thread and the `ProcessPoolExecutor` worker that performs the actual SIFT extraction (and on the way back, for the small keypoint/descriptor payload).

Backpressure and termination. The queue has a fixed maximum size (`queue_depth`, set to two items by default), which causes the `queue.put()` method to block automatically if the producer runs faster than the consumer. This mechanism slows down the producer and limits the number of fully-extracted images held in memory at any time. Once all images are sent, the producer places a special "sentinel" value into the queue, allowing the consumer to recognize it and exit its loop before the producer thread is joined and the executors are shut down.

Combining task and data parallelism. This variant layers a second, orthogonal form of parallelism on top of the producer-consumer overlap. The warping and blending step invokes `warp_and_blend_tiling`, which partitions the warped canvas into horizontal strips and blends them concurrently on a `ThreadPoolExecutor`. The producer-consumer overlap is *task* parallelism (different pipeline stages executing at the same time) while the tiled blend is *data* parallelism (a single operation split across a partition of its input). Because both the process pool (extraction) and the thread pool (blending) can be active simultaneously, they compete for the same physical cores.

Listing 3: Producer thread: extraction dispatch and handoff via a bounded queue.

```
def _producer(images, process_executor,
    out_queue):
    # Dispatch every extraction job to the
        warm process pool at once
    futures =
        [process_executor.submit(_extract_worker,
        img)
               for img in images]

    for i, fut in enumerate(futures):
        # Blocks until image i's extraction
            result is ready, in order
        kp_serialized, des = fut.result()
        kp = _deserialize_kp(kp_serialized)

        # Hands off a reference, not a copy:
            producer and consumer
        # share the same process. Blocks if
            the queue is full
        # (backpressure).
        out_queue.put((i, images[i], kp, des))

    # Signal completion to the consumer
    out_queue.put(_SENTINEL)
```

2.6. MapReduce Paradigm

Functional decomposition principles inspire this fourth architectural variant. Unlike the sequential, parallel, and producer-consumer pipelines discussed above, the MapReduce variant restructures the computation as a binary merge tree, directly targeting the bottleneck identified by Amdahl's law: in a linear fold, stitching image i can only begin once image $i-1$ has been merged, so the fold itself never parallelizes regardless of the number of available cores.

Map phase. The input dataset is first partitioned into n independent image indices, each dispatched to a worker process in a `ProcessPoolExecutor`. Because SIFT keypoint extraction operates on a single raw image with no dependency on any panorama state, this phase is *embarrassingly parallel*: all n extractions proceed concurrently, bounded only by the number of physical cores available. Each worker returns a self-contained *node*, consisting of the original image together with its serialized keypoints and descriptor matrix, ready to enter the reduction phase.

Reduce phase. The subsequent phase departs from linear accumulation entirely. Instead of a single running panorama, nodes are combined pairwise in a hierarchical reduction tree:

- **Level 1**: adjacent pairs $(0,1),(2,3),(4,5),\ldots$ are stitched concurrently, producing $\lceil n/2 \rceil$ partial panoramas.
- **Level 2**: pairs of partial panoramas from level 1 are stitched concurrently, producing $\lceil n/4 \rceil$ nodes.

  ... the process repeats until a single root node (the final panorama) remains.

Each processing level only waits for the level directly below it. Within the same level, however,

all image pairs are completely independent and can be processed in parallel by different worker processes. This creates a tree structure with only $\lceil \log_2 n \rceil$ steps, which is much faster than the $n-1$ steps required by the sequential method. If a level has an odd number of items, the leftover item is simply moved to the next level without being modified.

Atomicity of the reduce worker. A key implementation detail is that each pairwise merge executes as a single atomic unit inside one worker process (Listing 4). This is what allows the merged node to be fed back into the tree as an ordinary node at the next level, but it also means these four subphases cannot be individually timed from the orchestrating process; only the aggregate duration of the whole worker call is externally observable.

Listing 4: Reduce phase worker: an atomic merge unit

```
def _merge_pair_worker(args):
    cv2.setNumThreads(1)
    (img_a, kp_a_ser, des_a), (img_b,
        kp_b_ser, des_b) = args

    kp_a = _deserialize_kp(kp_a_ser)
    kp_b = _deserialize_kp(kp_b_ser)

    matches = match_features(des_a, des_b)
    if len(matches) < 4:
        return (img_a, kp_a_ser, des_a)  #
            degrade gracefully

    H = estimate_homography(kp_a, kp_b,
        matches)
    if H is None:
        return (img_a, kp_a_ser, des_a)

    try:
        merged = warp_and_blend(img_a,
            img_b, H)
    except ValueError as e:
        # Degenerate homography detected
            (oversized canvas):
        # keep the left node unchanged
            rather than crash the worker.
        return (img_a, kp_a_ser, des_a)

    sift = cv2.SIFT_create(nfeatures=8000)
    gray = cv2.cvtColor(merged,
        cv2.COLOR_BGR2GRAY)
    kp_m, des_m =
        sift.detectAndCompute(gray, None)
    kp_m_ser = [(p.pt, p.size, p.angle,
        p.response,
                 p.octave, p.class_id) for
                     p in kp_m]
    return (merged, kp_m_ser, des_m)
```

Correctness caveat. Unlike the linear-fold pipelines, this variant's output is *not* expected to be bit-identical to the sequential baseline. Composing homographies in tree order instead of left-to-right changes which images are warped relative to which, and perspective interpolation is inherently order-dependent; the resulting panorama can also differ in overall shape when the reduce worker's fallback discards a pair. The panorama produced is expected to remain geometrically plausible, but a pixel-level difference against the sequential baseline is not a meaningful correctness signal for this pipeline.

A structural trade-off. The tree-merge structure introduces a subtler risk that the linear fold does not share. In the linear fold, every new image is matched against the *entire panorama accumulated so far*, giving RANSAC a large, geometrically diverse pool of keypoints to draw from. In the MapReduce tree, by contrast, each pair is fixed *by position* in the index sequence, not by verified spatial adjacency: if two images assigned to the same tree node do not in fact overlap in the scene, no fallback within the algorithm itself can discover an alternative pairing. Empirically, this trade-off manifests as a substantially higher variance in the speedup achieved across image windows compared to the other three parallel pipelines: while the linear-fold variants oscillate in a narrow band, the MapReduce variant ranges from speedups exceeding $2\times$ down to occasional slowdowns below $1\times$ when a fixed pairing turns out to be geometrically unfavorable.

2.7. Joblib Implementation

This fifth variant replaces the manual `ProcessPoolExecutor` orchestration used in `parallel.py` with the high-level

`joblib.Parallel` interface, retaining the same overall architecture while delegating process-pool lifecycle management to Joblib's `loky` backend.

Motivation. Every parallel and MapReduce variant discussed so far pays an inter-process communication (IPC) cost whenever a NumPy image array crosses a process boundary: `multiprocessing` pickles the full array and copies it into the receiving worker's address space. For a half-resolution frame ($\approx$ 1.5 MB), this cost is paid once per image on every call to the extraction phase, including every warm-up and timed repetition of the benchmark. Rather than manage a `multiprocessing.shared_memory` block by hand (the approach taken by the shared-memory variant discussed in Section 2.8), this implementation delegates the problem to Joblib's built-in automatic memmapping.

Automatic memmapping. When an array argument passed through `Parallel(...)(delayed(fn)(arg) for arg in ...)` exceeds the `max_nbytes` threshold, Joblib transparently writes it to a temporary memory-mapped file on disk and passes only the file handle and shape/dtype metadata to each worker, which maps it back into memory read-only with no additional copy. The threshold used here, `max_nbytes="1M"`, is deliberately set well below the size of a single image frame (which is roughly 6-8 MB). This ensures that every large image automatically triggers memmapping, while small data payloads remain standard in-memory objects, avoiding unnecessary disk operations for tiny variables.

Listing 5: Joblib-based parallel Map phase

```
_MEMMAP_THRESHOLD = "1M"
_BACKEND = "loky"

def extract_features_joblib(images):
    results = Parallel(
        n_jobs=NUM_CORES,
        backend=_BACKEND,
        max_nbytes=_MEMMAP_THRESHOLD,
        prefer="processes",
    )(delayed(_extract_worker)(img) for
        img in images)
    # ... deserialize keypoints, return
        (kp_list, des_list, extraction_time)
```

Backend and worker-pool reuse. The `loky` backend is explicitly chosen using `backend="loky"` and `prefer="processes"`. This prevents Joblib from silently switching to a threading backend, ensuring that the CPU-heavy SIFT extraction uses separate processes and truly bypasses the GIL. Furthermore, `loky` automatically keeps its worker processes alive in the background across different calls. When `extract_features_joblib` is called repeatedly for different image windows, it reuses the same underlying processes instead of creating new ones from scratch every time. This avoids the high cost of spawning operating system processes, achieving the same efficiency as managing a long-lived `ProcessPoolExecutor` manually, but without the complex lifecycle code.

2.8. Shared Memory Optimization

Inter-process communication (IPC) serialization is a major performance bottleneck in Python's multiprocessing model. By default, when a NumPy image array crosses a process boundary, the parent process pickles it and the receiving worker reconstructs it, copying the entire array for every transfer. For a full-resolution frame ($\approx 4896 \times 3672 \times 3$ bytes, ≈ 54 MB), this copy cost is paid for every image during the MAP phase. In the MapReduce variant, this penalty occurs for every pair at every REDUCE level, where the transferred payload is an entire partial panorama instead of a single frame.

Strategy. This variant sidesteps the problem using `multiprocessing.shared_memory`, which allocates a contiguous block of raw bytes directly in the operating system's shared memory space rather than pickling data through

the multiprocessing IPC channel. After `load_images_parallel()` produces the window's image list, each array is copied *once* into its own `SharedMemory` block; workers then receive only a lightweight descriptor tuple (`shm_name, shape, dtype`) instead of the array itself, and reconstruct a NumPy view over the shared buffer with `np.ndarray(shape, dtype=dtype, buffer=shm.buf)`.

Listing 6: Allocating shared memory once for the whole window

```
def _alloc_shm_for_images(images):
    blocks, descriptors = [], []
    for img in images:
        shm = SharedMemory(create=True,
            size=img.nbytes)
        buf_arr = np.ndarray(img.shape,
            dtype=img.dtype, buffer=shm.buf)
        np.copyto(buf_arr, img)          #
            one-time copy into shared RAM
        blocks.append(shm)
        descriptors.append((shm.name,
            img.shape, img.dtype.str))
    return blocks, descriptors
```

What is actually eliminated. It is important to state precisely which direction of communication this optimizes. Only the *outbound* transferf (from the main process to each worker) is avoided: a worker no longer receives a pickled copy of the full image, only a small tuple of metadata. The *inbound* direction is unchanged: extracted keypoints and descriptors are still returned to the main process through the ordinary `ProcessPoolExecutor` return channel and pickled as usual, since they are orders of magnitude smaller than a raw frame and not worth placing in shared memory themselves.

Not a genuine zero-copy read, by design. A closer look at the worker function reveals that the read phase is not a strict zero-copy operation. Once connected, the worker explicitly copies the shared array into its local memory and immediately releases its connection to the shared block. This is a deliberate safety measure, not an oversight. If a worker kept the shared block open during the entire (and potentially slow) SIFT computation, the main process might try to clean up the memory (`shm.unlink()`) while the worker is still attached, causing the system to hang or crash. Making a quick local copy and releasing the connection keeps the shared memory's lifecycle short and safe, at the acceptable cost of one local memory copy per worker.

3. Experimental Setup

To ensure a statistically sound and reproducible comparison across the six stitching pipelines described in the previous sections, a dedicated benchmarking framework (`benchmark.py`) was developed. Rather than timing each pipeline in isolation with ad-hoc scripts, every pipeline is wrapped in a uniform interface and driven through the same sliding-window protocol, the same warm-up and repetition discipline, and the same statistical post-processing, so that any observed difference in wall-clock time can be attributed to the pipeline's architecture rather than to inconsistencies in how it was measured.

3.1. Hardware and Software Environment

All benchmarks were executed on a Windows machine equipped with an **Intel Core i7-8565U CPU @ 1.80 GHz** (4 physical cores, 8 logical threads) and **16 GB** of RAM, running the CPython 3.14 interpreter.

To maximize resource utilization, the framework is configured with the `NUM_CORES` constant set to match the logical thread count of the evaluation hardware. This parameter strictly governs the worker count of every `ProcessPoolExecutor` and `ThreadPoolExecutor` instantiated throughout the experiments, as well as the number of horizontal tiles generated during the data-parallel blending phase.

3.2. Pipeline Abstraction

Each of the six variants (sequential, parallel, producer-consumer, MapReduce, Joblib, and

shared-memory) is described by a lightweight `PipelineSpec` record (Listing 7): a display name, a `run(images, resources, seed)` callable returning a dictionary of per-phase timings plus the resulting panorama, and two boolean flags declaring whether the pipeline requires a process pool, a thread pool, or both. Before benchmarking a given window, the framework inspects every selected `PipelineSpec` and opens the *union* of executors required by any of them exactly once, reusing the same pools across the warm-up passes and all timed repetitions of every pipeline in that window. This design avoids conflating executor spawn overhead with the architectural cost the benchmark is actually meant to isolate.

Listing 7: The pipeline abstraction driving the benchmark

```
@dataclass
class PipelineSpec:
    name: str
    run: Callable[..., dict]
    needs_process_pool: bool = True
    needs_thread_pool: bool = False

pipelines_to_run = [
    SEQUENTIAL_SPEC,        # baseline
    PARALLEL_SPEC,
    PRODUCER_CONSUMER_SPEC,
    MAPREDUCE_SPEC,
    JOBLIB_SPEC,
    SHM_SPEC,
]
```

By convention, the first entry in the pipeline list is always the *baseline* (`sequential`); every subsequent entry is a *candidate* reported and exported against it, mirroring the sequential-vs-parallel comparison strategy used throughout this study.

3.3. Sliding-Window Protocol

Because datasets can contain too many images to stitch into a single panorama, the benchmark operates on non-overlapping sliding windows of `WINDOW_SIZE = 4` images.

Images for each window are loaded from disk exactly once and kept in memory. They are reused unchanged across every warm-up pass and timed repetition of all six pipelines. Since no pipeline mutates the input images in place—each allocating a new panorama via `warp_and_blend`, this approach carries no correctness risk and completely eliminates disk I/O variance from the timed measurements.

This design deliberately departs from a cold-cache protocol, where inputs would be reloaded before every single run. With six pipelines and multiple windows, avoiding repeated disk access drastically reduces the total benchmark duration.

3.4. Warm-up, Repetitions, and Thermal Management

For every pipeline, two untimed warm-up passes are executed first. This allows the underlying system and worker pools to reach a steady state before any measurements begin. After the warm-up, the benchmark records `N_RUNS = 3` independent repetitions.

To ensure consistent results, the framework forces an explicit garbage collection (`gc.collect()`) and pauses briefly between runs and between different pipelines. This approach clears leftover memory overhead and provides a short cooldown window for the CPU. By doing so, it prevents random garbage-collection pauses and hardware thermal throttling from skewing the final performance metrics.

3.4.1 Statistical Methodology

For every measured phase, the mean, standard deviation, and 95% confidence interval are computed using Student's *t*-distribution, consistent with the small-sample regime imposed by `N_RUNS = 3` (2 degrees of freedom, two-tailed critical value $t = 4.303$ at $\alpha = 0.05$). Speedup between the baseline and a candidate is then computed as the ratio of their means, with its own confidence interval propagated via the delta method, combining each side's coefficient of variation:

$$\frac{\sigma_S}{S} \approx \sqrt{\left(\frac{\sigma_{\text{base}}}{\mu_{\text{base}}}\right)^2 + \left(\frac{\sigma_{\text{cand}}}{\mu_{\text{cand}}}\right)^2}.$$

Because a pipeline may complete fewer than `N_RUNS` repetitions in a given window (see Sec-

tion 3.7), the baseline and candidate samples are permitted to differ in size; each side's own sample size is used for its variance term, while the more conservative (smaller) of the two sample sizes is used to look up the critical t-value, avoiding an overly narrow interval when one side has fewer observations than the other.

3.5. Handling Non-Measurable Phases

Not every pipeline can report a clean, standalone duration for every phase of the stitching pipeline. In the producer-consumer variant, extraction is deliberately overlapped with the rest of the chain and has no isolated duration; in the MapReduce variant, matching, homography, warping, and re-extraction all execute atomically inside a single worker call per tree node and can't be timed separately from the orchestrating process. To handle this, such unmeasurable phases are directly recorded as `None`. This ensures they are explicitly excluded from all aggregate statistical computations, and they are neatly rendered as `"n/a"` in both the console reports and the exported CSV files.

3.6. Correctness Validation

After the timed runs for a window complete, the last panorama produced by each candidate is compared against the baseline on a pixel-by-pixel basis. The absolute per-pixel difference is computed using `int32` arithmetic to avoid `uint8` underflow. The framework evaluates the maximum and mean differences, the percentage of identical pixels, the per-channel BGR breakdown and the Peak Signal-to-Noise Ratio.

A qualitative verdict is then assigned: *bit-identical* when the maximum difference is zero; *equivalent* when the difference is at most one least-significant bit due to floating-point rounding variations; or *visually identical* when the Peak Signal-to-Noise Ratio exceeds 40 dB. Outcomes failing these criteria are flagged for manual inspection. Shape mismatches, which are expected for the MapReduce variant, are reported as distinct outcomes to avoid meaningless pixel comparisons.

3.7. Robustness and Fault Tolerance

The benchmarking framework implements two safeguards to prevent isolated errors from invalidating the entire execution.

First, each call to the warping and blending routine is wrapped in a `try/except` block. This block catches a `ValueError` which is triggered if a degenerate homography attempts to create an oversized canvas. The framework defines this as any canvas exceeding four times the combined input area. When this occurs, the problematic image is skipped to prevent the system from exhausting its memory.

Second, all warm-up passes and timed repetitions are individually protected by exception handlers. If a pipeline fails during its warm-up or across all its timed runs within a specific window, it is simply excluded from that window's results. If the baseline pipeline fails, the framework immediately aborts the remainder of that window since comparisons are no longer possible. However, all previously recorded results for other windows and pipelines remain safely preserved.

3.8. Output Artifacts

For each window, the framework exports two CSV files under `results/`: `benchmark_results.csv`, containing per-phase mean, standard deviation, margin, and speedup with confidence bounds for every (baseline, candidate) pair and `correctness_results.csv`, containing the pixel-comparison metrics described above. The last panorama produced by every pipeline in every window is additionally saved to `data/output/benchmark/` for visual inspection, independent of the numerical results.

References

[1] OpenDroneMap Contributors. Aukerman Dataset. https://github.com/OpenDroneMap/odm_data_aukerman, 2016.